Software Engineering

Assignment #03


WeLcome

Agenda: Assignment #03 - Software Engineering

1. Assignment #03
2. References
3. Submission details
4. Q & A

Agenda: Assignment #03 - Software Engineering

1. **Assignment #03**
2. References
3. Submission details
4. Q & A

Assignment details

- AI-assisted coding, vibe coding and prototype cleanup.
- Scenario: Smart Campus Room Availability Checker.
- Work individually. Build a working one-page prototype and document the engineering decisions.
- Use at least one AI assistant and preserve evidence of your own review.

Learning outcomes

- Translate a user story into a precise, testable contract.
- Use an AI coding assistant to implement and review a small function.
- Use Bolt.new to generate a first web prototype and improve it.

continues …

Learning outcomes

- Explain how agent tools, permissions, and human checkpoints affect dependability.
- Demonstrate tests, cleanup and evidence-based verification.

Scenario and scope

- User story: As a student, I want to check a room’s availability so that I can choose a valid booking time.

- Build one responsive page for one selected room, with fictional booking data.
- The page checks availability only; it does not create or confirm bookings.
- Exclude login, database, backend, external API, payments and unrelated features.

Required rules: AC1–AC3

- AC1. Candidate times are integer minutes after midnight: 0 <= start < end <= 1440; start must be strictly after now.
- AC2. Duration must not exceed 120 minutes. Exactly 120 minutes is permitted.
- AC3. Reject every request when the selected room is blocked.

- Assume now is valid (0–1439); use fictional data only.

Required rules: AC4–AC6

- AC4. Reject overlap with any existing booking. Intervals touching at an endpoint do NOT overlap.
- AC5. Display an unambiguous Available / Unavailable result; do not mutate existing bookings.
- AC6. Provide understandable validation feedback and a usable narrow/mobile layout.

- Treat these criteria as fixed: reject AI suggestions that change their meaning.

Task 1: baseline and plan

- Before prompting AI, write the scope and acceptance criteria in your own words.
- Identify at least two likely failure points (for example, endpoint overlap and starts-now).
- Describe the function contract: can_book(start, end, now, blocked, existing).
- Record your initial assumptions and what will be verified manually.

Task 2: AI-assisted Python implementation

- Create booking.py and test_booking.py with Python 3.
- Ask AI to propose a short plan, then generate can_book(...) using only the standard library.
- Save the first generated implementation as v1 before fixing any errors.
- Review each conditional branch and explain how it satisfies AC1–AC5.
- Keep inputs unchanged and return True only when the request is valid.

Task 3: prototype v1 with Bolt.new

- Prompt Bolt.new to build the one-page Smart Campus Room Availability Checker.
- Include AC1–AC6 and explicitly forbid login, backend and database.
- Save the exact prompt, screenshot, tool/model details if visible, and first usable preview.
- Identify main generated files/components and inspect their role.

- Do not treat v1 as the final submission.

Task 4: execute test cases

- Base data: now=540; blocked=False; existing=[(600,660)].
- Touching: 660–720 -> Available. Overlap: 630–690 -> Unavailable.
- Blocked: 660–720, blocked=True -> Unavailable.
- Exactly 120 min: 720–840 -> Available; 121 min: 720–841 -> Unavailable.
- Starts now: 540–570 -> Unavailable.

- Also test reversed times, day bounds, multiple bookings and unchanged inputs

Task 5: cleanup and retest

- Record at least two concrete findings from v1 or justify preventive cleanup if all tests pass.
- Prioritize a functional defect and a usability/code-quality issue where available.
- For each issue: evidence -> focused AI prompt -> change -> retest.
- Avoid generic “make it better” prompts and unrequested feature additions.

- Run the complete acceptance test set again on the final version.

Task 6: coding-agent analysis

- Draw a simple agent loop: user goal -> model/plan -> tool call -> observation -> next step -> human review.

- Identify what context, tools and permissions your assistant or Bolt.new used or appeared to use.

- Distinguish observed behavior from assumptions about internal implementation.

continues …

Task 6: coding-agent analysis

- Explain one safety checkpoint before executing or accepting AI-generated changes.
- Relate the workflow to system reliability, security and human responsibility.

Required AI workflow

- A. Plan: ask for checks, assumptions and boundary tests; approve the plan yourself.
- B. Generate: save Python v1 and web prototype v1.
- C. Critique: request a review against AC1–AC6; verify each claim independently.

continues …

Required AI workflow

- D. Repair: use small, targeted prompts; record exactly what changed.
- E. Verify: rerun tests, document remaining limitations and explain decisions.
- Save at least four meaningful prompt-response exchanges.

Evidence and traceability

- Prepare a table: AC ID | implementation/file | test input | expected | v1 actual | final actual.

- Keep an issue log: issue | reproduction | priority | prompt/change | retest result.

- Include before/after screenshots for at least one meaningful improvement.

continues …

Evidence and traceability

- Record tool name, model if displayed, and date of AI use.
- Never claim a test passed unless it was actually executed.

Reflection: dependability and AI

- In 100–200 words, compare AI-assisted coding with vibe coding in this assignment.
- Explain one example each of reliability, availability/usability and security risk.
- Discuss why code review, tool permissions and tests remain human responsibilities.

continues …

Reflection: dependability and AI

- Identify one AI recommendation you rejected or modified, and why.

- Describe what you would improve next without expanding the current scope.

Agenda: Assignment #03 - Software Engineering

1. Assignment #03
2. **References**
3. Submission details
4. Q & A

References details

- Lesson #05 - Software Engineering - Lecture: dependable systems, security, reuse and software engineering.

- Lesson #05 - Software Engineering - Practice: AI-assisted coding and can_book tests.

References details

- Lesson #06 - Software Engineering - Lecture: coding agents, agent loop, tools and harness design.
- Lesson #06 - Software Engineering - Practice: vibe coding, Bolt.new and prototype cleanup.
- Reference baseline: Assignment #02 format and evidence-driven AI workflow.

Agenda: Assignment #03 - Software Engineering

1. Assignment #03
2. References
3. **Submission details**
4. Q & A

Submission package

- A3_Surname_Name_Report.pdf: scope, criteria, traceability, tests, issues, agent analysis and reflection.

- booking.py and test_booking.py: readable source and runnable tests.

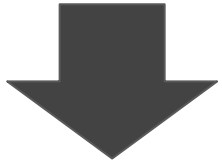
continues …

Submission package

- Prototype: Bolt.new project link if available, plus screenshots and source/export if available.

- AI_Evidence.pdf: at least four prompts with responses, review decisions and revisions.

Submission details



Please send email to **kibo.kz.contact.01@gmail.com** indicating
Software Engineering - Assignment #03 - Your Surname & Name

Example of topic of message:
Software Engineering - Assignment #03 - Aldamuratov Jomart



Submission details

DEADLINE: October 19, 2026 23:59:59



Submission details

Please do not send your work to z.aldamuratov@kbtu.kz (even in carbon copy [cc]), because it is creating double work and confusion. Teaching Assistants check kibo.kz.contact.01@gmail.com account while I check z.aldamuratov@kbtu.kz. We may confuse and may ignore your work by trusting each other.



Agenda: Assignment #03 - Software Engineering

Q & A